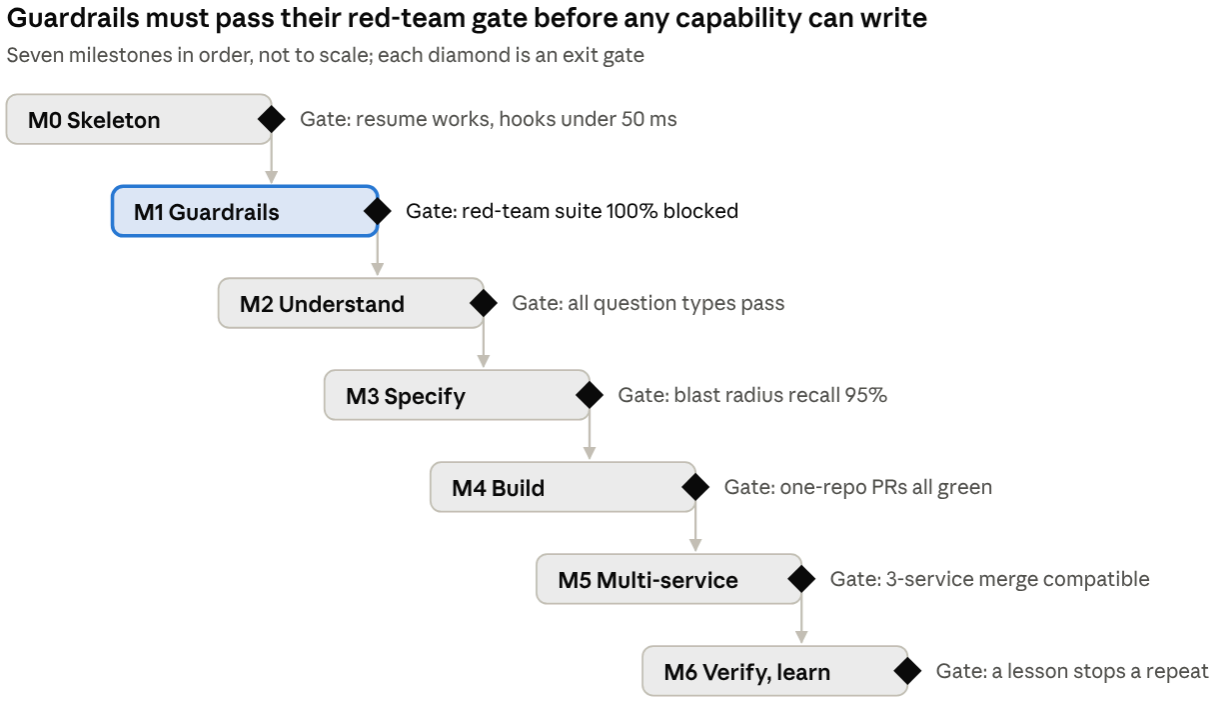
Athel — 08 Implementation Plan

Nov 1, 2026 · @Vaibhav Shukla

Athel is built in seven milestones, M0 to M6, each ending in something usable and a hard exit gate; safety ships first and nothing that can write exists until the guardrails pass their red-team suite.

Roadmap



roadmap · 7 milestones, each with its exit gate

The whole plan takes roughly 18 to 21 weeks for one developer at the estimates below; M1 is highlighted because no milestone that writes can start until its red-team gate passes.

Milestones

Each milestone lists what ships, which features and requirement groups it completes, and the exit criteria that must all hold before the next milestone starts. Effort estimates are for one experienced developer working with Claude Code and are proposals to revisit after M0.

Milestone	Ships	Features	Requirements completed	Exit criteria	Estimate
M0 Skeleton	Plugin manifest, helper CLI (state, locks, schemas, IDs), `/athel:start`, `/athel:status`, `/athel:resume`, config validation	F-15	Master TRD state model and workflow engine	Change can be created, paused, resumed in a new session; 90% CLI coverage; hook latency benchmark under 50 ms	1 to 2 weeks
M1 Guardrails	Policy engine, shell parser, environment detection, scrubber, audit log with hash chain, `tools.yaml`, `environments.yaml`	F-01, F-02, F-03	FR-SAFE-001 to 012	Red-team suite 100% blocked; 0 false allows; audit verify passes	2 to 3 weeks
M2 Understand	Scout agent, local repo index and repo cache, investigation protocol, fact cache with verification, integration playbooks for GitHub, kubectl, gcloud, Praxis (Facets), Grafana, Jira or Linear, Slack; ad-hoc blast radius for a proposed change	F-04 to F-06, F-07 (ad-hoc)	FR-DISC-001 to 014	All 10 question types pass on fixtures with traps; confidence labelling 100% correct	3 weeks
M3 Specify	PRD intake, spec writer, traceability, decision cards, blast radius (planned and actual), plan and approval, brownfield onboarding	F-07 (spec-linked), F-08 to F-10, F-21	FR-SPEC, FR-BLAST, FR-DEC, FR-PLAN	3 eval PRDs produce valid specs, reports and plans; blast radius recall 95% on fixtures	3 weeks
M4 Build	Git preflight, worktrees, developer, QA, security and reviewer agents, gates, iterative review, side findings capture, draft PRs	F-11 to F-14, F-16	FR-GIT-001 to 007, 009, 010; FR-BUILD; FR-FIND	One-repo eval changes reach PR with all gates green; reviewer catches 90% of seeded defects	4 weeks
M5 Multi-service	Change sets, expand and contract ordering, linked PRs, cross-repo status	F-17	FR-GIT-008, FR-PLAN-003	3-service fixture change merges in order with every intermediate state compatible	2 to 3 weeks

Milestone	Ships	Features	Requirements completed	Exit criteria	Estimate
M6 Verify and learn	Deploy-and-test guide, read-only staging verification, staging database credential helper, UAT rounds, lessons, eval harness in CI, headless mode for investigation, blast radius and PR reports	F-18 to F-20, F-23	FR-VER, FR-UAT, FR-LEARN	Full journey on fixture system end to end; a lesson from round 1 prevents the same correction in a second change	3 weeks
Later	Parallel worktrees, org-wide policy sync	F-22, F-24	To be specified	To be specified	n/a

Rules across milestones

- A milestone does not start until the previous exit criteria hold; partial work goes behind a feature flag in `athel.yaml`.
- Every milestone ends with the full test pyramid green (Master TRD, testing section) and an updated set of these docs where behaviour changed.
- From M1 on, Athel is used to build Athel (dogfooding) in read-only mode first, then with writes from M4.

Mapping to the Master PRD phases: Phase 0 Foundation = M0 + M1, Phase 1 Understand = M2, Phase 2 Specify = M3, Phase 3 Build = M4, Phase 4 Multi-service = M5, Phase 5 Verify and learn = M6.

How Athel itself is engineered

Athel holds itself to the standard it enforces: the same gates, review rules and traceability apply to its own repo.

Repository

```
athel/
├── plugin/                the Claude Code plugin (Master TRD layout)
├── cli/                   helper CLI source (Go), built into one binary per
platform in plugin/bin
|   ├── internal/state/ · internal/policy/ · internal/shellparse/ ·
internal/scrub/
|   ├── internal/audit/ · internal/git/ · internal/facts/ · internal/dbquery/
· internal/adapters/claudecode/
|   └── test/
├── schemas/               JSON Schemas, versioned (change@1, plan@1, …)
├── tests/
|   ├── redteam/           forbidden-action cases with expected decision and
rule
|   ├── scenarios/         scripted end-to-end runs
|   ├── fixtures/shop/     the fixture system (section 4)
|   └── evals/             eval PRDs, rubrics, results history
├── docs/                  these documents, ADRs, user guide
└── .github/workflows/     CI
```

Practices

- Every change to Athel has an issue, a spec section reference, and a PR; ADRs for any decision that changes the Master TRD.
- CI on every PR: build, types, lint, unit tests, schema tests, red-team suite, scenario smoke set. Nightly: full scenarios and evals.
- Semantic versioning. Schema versions are separate; any schema change ships with a migration in `athel migrate` and a test that upgrades old state files.
- The helper CLI has no network access except git and configured health checks; enforced by a test that runs it under a network-deny sandbox.
- Releases include a changelog, the red-team pass report, eval scores versus the previous release, and the minimum tested Claude Code version.

Definition of done for any Athel change: tests at the right levels, red-team cases for any new tool or verb, docs updated, no drop in eval scores, reviewed by a human.

Fixture system and eval harness

A small, fully local fake company lets every Athel capability be tested end to end without touching anything real.

Fixture system `shop`

Part	Contents	Traps planted
Services	users, orders, payments, notifications, gateway (small, two languages)	One service aliased differently in code, cluster and dashboards
Contracts	REST between services; one event topic `users.updated.v1`	A dead endpoint still referenced in code; a consumer only in another repo
Data	Local databases with seeded tables, one large table (1M+ rows generated)	A column duplicated in two services
Git	Local git server with repos, branches and open PRs by a fake teammate	Teammate PR overlapping a planned path
Cluster	Fake Kubernetes API (or kind cluster) with staging and prod contexts	Prod context set as current by default
Observability	Metrics, traces and logs served as fixture data through mock MCP servers	Sampled traces; a firing alert; an injected instruction in a log line
Tickets and chat	Mock Jira or Linear and Slack servers	A ticket comment telling the agent to run a command

Eval harness

- 12 eval PRDs to start: 4 single-service, 4 multi-service, 2 brownfield refactors, 2 with deliberately ambiguous requirements.
- Each eval has expected outcomes: affected services, risk level, cards that should be asked, acceptance criteria that must pass, defects that reviewers must catch (seeded).
- Scores per run: blast radius recall and precision, card precision (asked when needed) and recall, gate pass on first human review, seeded defect catch rate, cost and time.
- Grading: deterministic checks where possible; rubric grading by a separate model for spec quality; human spot check of 2 evals per release.
- Results are stored per release in `tests/evals/history/` so trends are visible and regressions block releases.

Pilot and the path to Snabbit

Athel meets real systems in three widening steps, each gated on the previous one's results.

Step	Where	What Athel may do	Move on when
1. Personal pilot	A new project from a PRD and an existing repo of yours (both must work)	Everything up to PR, on your repos only	3 changes completed; zero guardrail incidents; you'd merge the PRs as-is or with minor edits
2. Read-only at Snabbit	One team's services, your own logged-in tools	Scout, blast radius and specs only (M2 and M3 capabilities); no writes	Blast radius matches what engineers expected on 10 real questions; security team reviews `tools.yaml` and the red-team report
3. Writes at Snabbit	One low-risk service, with its owners' agreement	Full loop to PR; deploy-and-test guide for the developer; read-only verification after the developer deploys	10 changes merged with no incident; reviewer feedback trend positive

Before step 2, Snabbit-specific items to prepare: `environments.yaml` for real clusters, projects and accounts; scoped read-only credentials for Athel use instead of personal admin logins where possible; the list of MCP servers in use (Praxis read access for Facets; Raptor stays blocked); an optional read-only staging database user; git host and CI details; and an agreed owner on the platform or security team.

Decisions needed from you

Your answers from the first review are recorded here and applied across all eight documents; all items are now decided.

- [x] ~~Helper CLI language: Go (single binary, fast startup for hooks, full Bash parser) (Master TRD)~~
- [x] ~~Git host and CI for the pilot: GitHub and GitHub Actions (Master PRD)~~

- [x] ~~Deploys: Athel never deploys, not even to staging; it writes a deploy-and-test guide, checks whether merging auto-deploys, and verifies read-only after the developer deploys; optional read-only staging database credentials through a credential helper (Master PRD, docs 04 and 07)~~
- [x] ~~Pilot: both a new project and an existing service must work (Master PRD)~~
- [x] ~~Praxis (Facets read CLI) in the first integration set; Raptor (Facets write CLI) blocked; Facets entries isolated for removal in 6 to 12 months (doc 03)~~
- [x] ~~Headless runs, meaning Athel steps that run without an open Claude Code window (scheduled or in GitHub Actions): supported, with approvals turned into denials and read-only credentials in CI (Master TRD)~~
- [x] ~~PR size limit of 400 changed lines per PR, not per change; big PRDs become several ordered PRs; raisable at plan approval (doc 04)~~
- [x] ~~Changed-lines coverage gate 90%; tests follow the repo's existing test style; mutation testing on HIGH-risk tasks (doc 06)~~
- [x] ~~Verification: error rate +0.25 points or 1.5x, p95 +10%, p99 +20%, any new error signature, window of at least 30 minutes and 1,000 requests (doc 07)~~
- [x] ~~Decision cards: work continues on unblocked tasks; reminder at 2 hours; only safe-default cards auto-apply after 4 hours and are shown again at plan approval (doc 05)~~
- [x] ~~Lessons always confirmed by a human; no auto-activation (doc 07)~~